

به نام خدا



پروژه درس جبر خطی - ترم ۴۰۴۲

Indexing Semantic Latent

استاد : دکتر نیکان

اعضای گروه: علی کارخانه - مصطفی شیبانی - حسین نادری - مرتضی رحیمی

سوالات عملی:

سوال ۷ – بارگذاری و پیش‌پردازش متن (حذف علائم نگارشی و کوچک‌سازی حروف)

در پروژه‌های بازیابی اطلاعات و نمایه سازی پنهان متن (LSI)، پیش‌پردازش متن یکی از حیاتی‌ترین مراحل است. هدف از این کار، حذف اطلاعات بیهوده (Noise) و یکدست‌سازی کلمات است تا محاسبات برداری دقیق‌تر انجام شوند. کد زیر فرآیند تمیزکاری متون را انجام می‌دهد:

```
1 # =====
2 # Q7 - Load + preprocess (lowercase, remove punctuation)
3 # =====
4 def preprocess(text):
5     text = text.lower()
6     text = re.sub(r'[^a-z\s]', ' ', text)
7     text = re.sub(r'\s+', ' ', text).strip()
8     return text
9
10 def q7_load_and_preprocess():
11     df = pd.read_csv(DATA_PATH)
12     print("Q7 | Dataset shape:", df.shape)
13     df['clean_text'] = df['Text'].apply(preprocess)
14     return df
```

توضیح عملکرد متدها و توابع:

۱. تابع `preprocess(text)`: این تابع یک متن خام را به عنوان ورودی دریافت کرده و مراحل زیر را روی آن اعمال می‌کند تا متنی یکدست ایجاد شود:

- **تبدیل به حروف کوچک (`text.lower()`)**: تمام حروف بزرگ انگلیسی به حروف کوچک تبدیل می‌شوند. این کار باعث می‌شود کلماتی مانند "Data" و "data" یکسان دیده شوند و ابعاد ماتریس نهایی بیهوده بزرگ نشود.
- **حذف کاراکترهای غیرالفبایی**: با استفاده از عبارات منظم (Expressions Regular)، هر کاراکتری که جزء حروف الفبای انگلیسی (a-z) یا فاصله (\s) نباشد، با یک فاصله معمولی جایگزین می‌شود. این کار علائم نگارشی نظیر ویرگول، نقطه، علامت سوال و همچنین اعداد را کاملاً حذف می‌کند.
- **حذف فاصله‌های اضافه**: حذف علائم نگارشی در مرحله قبل ممکن است چندین فاصله متوالی ایجاد کرده باشد. الگوی `\s+` تمام فاصله‌های متوالی را به یک تک‌فاصله تبدیل می‌کند و متد `strip()` نیز فاصله‌های خالی ابتدا و انتهای متن را پاک می‌کند.

۲. تابع `q7_load_and_preprocess()`: این تابع وظیفه مدیریت جریان داده را بر عهده دارد:

- ابتدا مجموعه داده را از مسیر `DATA_PATH` به صورت یک دیتافریم پاندا `df` بارگذاری می‌کند.
- با چاپ `df.shape`، ابعاد داده‌ها (تعداد کل اسناد و ویژگی‌ها) را جهت بررسی صحت بارگذاری نمایش می‌دهد.
- در نهایت با استفاده از متد `apply()`، تابع تمیزکاری متنی را روی تک‌تک سطرهای ستون `'Text'` اعمال کرده و نتیجه را در ستون جدیدی به نام `'clean_text'` ذخیره می‌کند.

۳. **خروجی اجرای تابع و تحلیل ساختار داده‌ها**: پس از فراخوانی تابع بارگذاری و پیش‌پردازش متنی، پیش‌نمایشی از دیتافریم خروجی به همراه ابعاد آن تولید می‌شود:

- **ابعاد مجموعه داده** (`Dataset shape: (2225, 2)`): خروجی نشان می‌دهد که دیتاست اولیه دارای ۲۲۲۵ سطر (سند متنی) و ۲ ستون اصلی است. پس از اجرای تابع و اضافه شدن ستون تمیزشده، تعداد ستون‌ها به ۳ افزایش می‌یابد (`2225 rows × 3 columns`).

• بررسی ساختار ستون‌ها:

۱. **ستون Text**: شامل متن خام اسناد است که در آن علائم نگارشی، کاراکترهای خط جدید (`\n`) و حروف بزرگ انگلیسی به همان شکل اولیه دیده می‌شوند.
۲. **ستون Label**: شامل برچسب‌های عددی (مثلاً دسته ۰ برای سیاست و دسته ۴ برای تجارت) است که دسته‌بندی موضوعی اسناد را مشخص می‌کند.

Q7 | Dataset shape: (2225, 2)

	Text	Label	clean_text
0	Budget to set scene for election\n\n Gordon B...	0	budget to set scene for election gordon brown ...
1	Army chiefs in regiments decision\n\n Militar...	0	army chiefs in regiments decision military chi...
2	Howard denies split over ID cards\n\n Michael...	0	howard denies split over id cards michael howa...
3	Observers to monitor UK election\n\n Minister...	0	observers to monitor uk election ministers wil...
4	Kilroy names election seat target\n\n Ex-chat...	0	kilroy names election seat target ex chat show...
...	...	...	...
2220	India opens skies to competition\n\n India wi...	4	india opens skies to competition india will al...
2221	Yukos bankruptcy 'not US matter'\n\n Russian ...	4	yukos bankruptcy not us matter russian authori...
2222	Survey confirms property slowdown\n\n Governm...	4	survey confirms property slowdown government f...
2223	High fuel prices hit BA's profits\n\n British...	4	high fuel prices hit ba s profits british airw...
2224	US trade gap hits record in 2004\n\n The gap ...	4	us trade gap hits record in the gap between us...

2225 rows x 3 columns

شکل ۱: خروجی نهایی دیتافریم پس از انجام عملیات پیش‌پردازش متنی

۳. **ستون جدید clean_text:** خروجی نهایی حاصل از اعمال تابع preprocess است. همان‌طور که در تصویر مشخص است، تمامی حروف بزرگ به حروف کوچک تبدیل شده‌اند، کاراکترهای کنترلی مانند \n و علائم نگارشی کاملاً حذف شده و کلمات صرفاً با یک تک‌فاصله از یکدیگر جدا شده‌اند تا برای پردازش‌های بعدی آماده باشند.

سوال ۸ – استخراج ۳۰ کلمه پرتکرار و رسم نمودار میله‌ای
در این بخش، هدف شناسایی و استخراج ۳۰ کلمه پرتکرار موجود در کل اسناد با امکان فعال یا غیرفعال‌سازی فیلتر کلمات توقف (Stopwords) است. کد مربوط به این بخش به صورت زیر است:

```

1 # =====
2 # Q8 - Top 30 most frequent words + bar chart
3 # =====
4 def q8_top30_words(df, fname, stopwords=False):
5     if stopwords:
6         stopwords = ENGLISH_STOP_WORDS
7     else:
8         stopwords = {}
9
10    stop_words_set = set(stopwords)
11
12    all_words = []
13    for text in df['clean_text']:
14        words = [w for w in text.split() if w not in stop_words_set and len(w) > 1]
15        all_words.extend(words)
16
17    counter = Counter(all_words)
18    top30 = counter.most_common(30)
19    print("\nQ8 | Top 30 frequent words:", top30)
20
21    words, counts = zip(*top30)
22    fig, ax = plt.subplots(figsize=(14, 6))
23    ax.bar(words, counts, color='steelblue')
24    plt.xticks(rotation=60, ha='right')
25    ax.set_title('Top 30 Most Frequent Words (all documents)')
26    ax.set_ylabel('Frequency')
27    plt.tight_layout()
28    % plt.savefig(f'media/{fname}.png', dpi=130)
29    plt.close(fig)
30
31    return top30

```

توضیح عملکرد متدها و توابع:

۱. **منطق شرطی کلمات توقف (stopwords و stop_words_set):** تابع در این نسخه منعطفتر طراحی شده است و پارامتر stopwords را به عنوان ورودی می‌پذیرد:

- اگر مقدار stopwords برابر با True پاس داده شود، کلمات توقف استاندارد انگلیسی (ENGLISH_STOP_WORDS) به عنوان فیلتر فعال می‌شوند.
- در غیر این صورت (حالت False)، یک دیکشنری خالی {} تخصیص داده می‌شود که به معنی عدم فیلتر کردن کلمات توقف است.
- کلمات توقف در نهایت به یک ساختار مجموعه (set) تبدیل می‌شوند؛ زیرا جستجو در ساختار مجموعه در پایتون از مرتبه زمانی $O(1)$ بوده و سرعت پردازش را بسیار افزایش می‌دهد.

۲. جمع‌آوری و فیلتر کلمات:

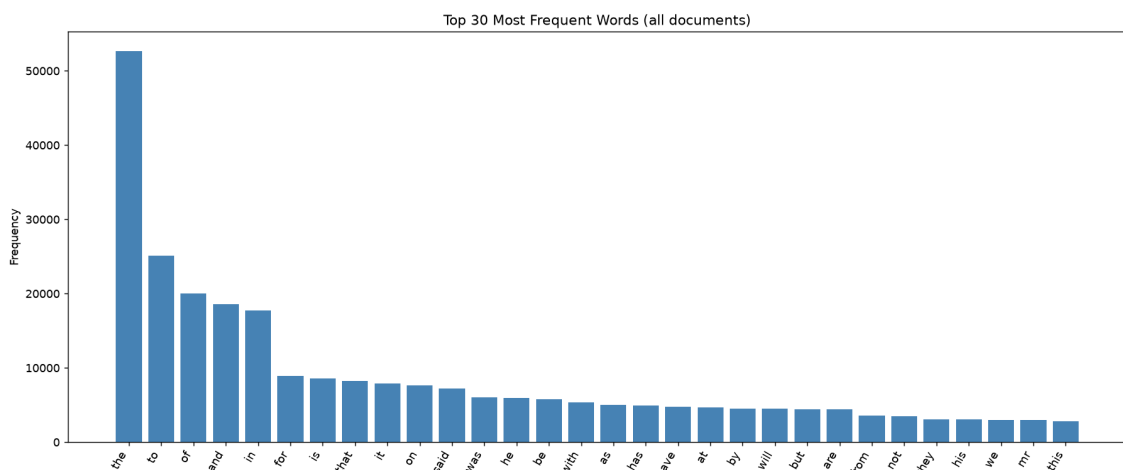
- با پیمایش ستون متون تمیزشده (clean_text)، هر متن با متد split() به کلمات سازنده‌اش شکسته می‌شود.
- کلماتی که در مجموعه‌ی کلمات توقف تعریف‌شده وجود نداشته باشند (w not in stop_words_set) و طول آن‌ها بزرگتر از یک حرف باشد ($\text{len}(w) > 1$)، فیلتر شده و به لیست نهایی اضافه می‌گردند.

۳. **شمارش و استخراج ۳۰ کلمه برتر:** کلاس Counter فرکانس تکرار کلمات را محاسبه کرده و با متد most_common(30)، ۳۰ کلمه با بیشترین تکرار را به صورت لیستی از توپل‌های (کلمه، فرکانس) باز می‌گرداند.

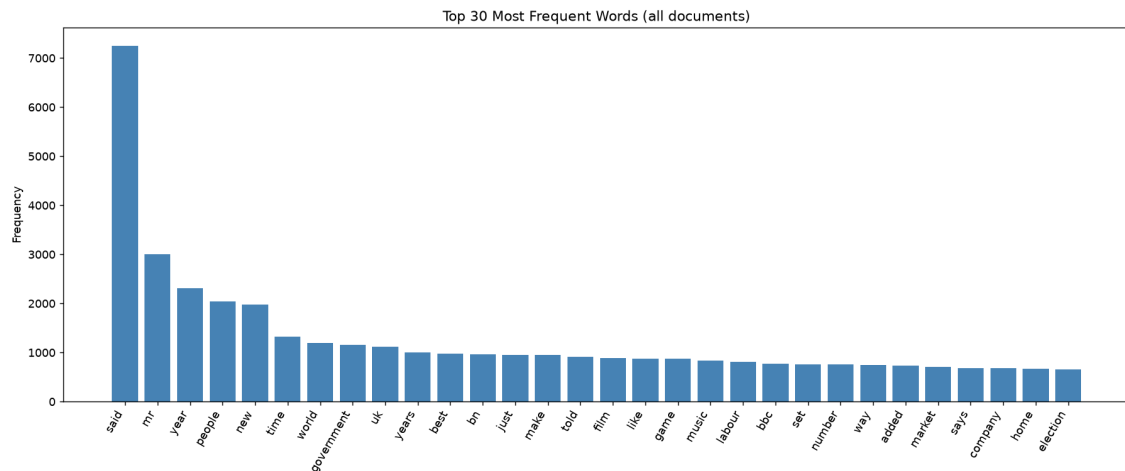
۴. رسم و مدیریت نمودار میله‌ای:

- با دستور zip(*top30) داده‌ها باز شده و به دو بخش کلمات و فرکانس‌ها تقسیم می‌شوند تا در نمودار میله‌ای به کار روند.
- نمودار میله‌ای با رنگ آبی فولادی (steelblue) رسم شده و برجسب‌های محور افقی جهت جلوگیری از هم‌پوشانی ۶۰ درجه چرخانده می‌شوند.
- نام فایل ذخیره‌سازی نمودار به صورت پویا با استفاده از متغیر ورودی fname تعیین شده است (که در کدهای فعلی جهت صرفه‌جویی در پردازش کامنت شده است) و در نهایت با plt.close() پنجره ترسیم بسته می‌شود تا حافظه آزاد گردد.

۵. **بررسی تجربی و مقایسه نمودارها (نقش حیاتی کلمات توقف):** برای درک عمیق‌تر تأثیر پیش‌پردازش، تابع فوق در دو حالت اجرا شده و نتایج آن در قالب دو نمودار زیر ذخیره و تحلیل شده است:



شکل ۲: ۳۰ کلمه پرتکرار بدون حذف کلمات توقف (غلبه کامل حروف ربط و اضافه)



شکل ۳: ۳۰ کلمه پرتکرار پس از حذف کلمات توقف (نماین شدن کلمات کلیدی موضوعی)

• **تحلیل حالت اول (بدون حذف کلمات توقف – شکل ۲):** همان‌طور که در نمودار اول مشاهده می‌شود، کلماتی مانند “the” (با بیش از ۵۰,۰۰۰ تکرار)، “to”، “of” و “and” کاملاً بر داده‌ها غلبه کرده‌اند. این کلمات صرفاً نقش ساختاری در زبان انگلیسی دارند و کوچک‌ترین اطلاعاتی درباره موضوع اصلی سند (مانند ورزشی، سیاسی یا علمی بودن آن) به ما نمی‌دهند.

• **تحلیل حالت دوم (با حذف کلمات توقف – شکل ۳):** در نمودار دوم، با فعال‌سازی پارامتر `stopwords=True` و فیلتر کردن این کلمات بیهوده، سقف فرکانس کلمات به زیر ۸,۰۰۰ تکرار کاهش یافته است. با حذف نویزهای زبانی، کلمات معنادار و موضوعی‌تری مانند “government” (دولت)، “film” (فیلم)، “game” (بازی)، “music” (موسیقی) و “election” (انتخابات) ظاهر شده‌اند که به ما در دسته‌بندی و تحلیل محتوایی اسناد کمک شایانی می‌کنند.

سوال ۹ – رسم ابر کلمات اختصاصی براساس وزن‌دهی TF-IDF

در این بخش، هدف نمایش بصری کلمات کلیدی اسناد با استفاده از یک ابر کلمات (Word Cloud) اختصاصی است. از آنجایی که پکیج پیش‌فرض `wordcloud` در حالت آفلاین در دسترس نبوده، پیاده‌سازی به صورت دستی و با استفاده از توانمندی‌های ترسیم کتابخانه `matplotlib` انجام گرفته است. معیار انتخاب کلمات، وزن مجموع فرکانس سند معکوس (TF-IDF) آن‌هاست. کد بهینه‌شده به صورت زیر است:

```
1 # =====
2 # Q9 - Word cloud (top 20 words by TF-IDF), custom-rendered
3 # (wordcloud package unavailable offline -> manual matplotlib rendering)
4 # =====
5 def q9_wordcloud(df, fname):
6     extra_stop = ['said', 'mr', 'told', 'year', 'years', 'new', 'time', 'best', 'world']
7     stop_words = list(ENGLISH_STOP_WORDS.union(extra_stop))
8
9     vectorizer = TfidfVectorizer(stop_words=stop_words, max_df=0.9, min_df=2)
10    tfidf = vectorizer.fit_transform(df['clean_text'])
11    scores = np.asarray(tfidf.sum(axis=0)).ravel()
12    vocab = vectorizer.get_feature_names_out()
13
14    top_idx = np.argsort(scores)[::-1][:20]
15    top_words = [(vocab[i], scores[i]) for i in top_idx]
16
17    random.seed(7)
18    fig, ax = plt.subplots(figsize=(12, 8))
19    ax.axis('off')
20
21    max_score = max(s for _, s in top_words)
22    min_score = min(s for _, s in top_words)
23    placed = []
24
25    def overlaps(x, y, w, h, placed):
```

```

26         for (px, py, pw, ph) in placed:
27             if abs(x - px) < (w + pw) / 2 * 0.75 and abs(y - py) < (h + ph) / 2 *
0.75:
28                 return True
29         return False
30
31     for word, score in top_words:
32         size = 16 + 55 * (score - min_score) / (max_score - min_score)
33
34         for _ in range(300):
35             x = random.uniform(0.1, 0.9)
36             y = random.uniform(0.1, 0.9)
37             w = len(word) * size * 0.011
38             h = size * 0.02
39             if not overlaps(x, y, w, h, placed):
40                 placed.append((x, y, w, h))
41                 break
42
43         color = plt.cm.tab20(random.random())
44         ax.text(x, y, word, fontsize=size, ha='center', va='center', color=color,
45               fontweight='bold', fontfamily='sans-serif', transform=ax.transAxes)
46
47     ax.set_title('Top 20 Words Wordcloud', fontsize=16)
48     plt.tight_layout()
49     % plt.savefig(f'media/{fname}.png', dpi=130)
50     plt.close(fig)
51
52     return top_words

```

توضیح عملکرد متدها و تغییرات جدید:

۱. **توسعه‌ی کلمات توقف (stop_words):** مجموعه‌ای از کلمات عمومی انگلیسی (ENGLISH_STOP_WORDS) با کلمات پر تکرار کم‌اهمیت دیگر که در متغیر extra_stop تعریف شده‌اند، ادغام (union) می‌شوند. این کار باعث تفکیک کلمات موضوعی باارزش از کلمات تکراری زبان می‌شود.

۲. **بردارسازی با معیار TF-IDF (TfidfVectorizer):** یک مدل بردار ساز با حذف کلمات توقف تعریف می‌شود. پارامتر max_df=0.9 کلماتی را که در بیش از ۹۰ درصد اسناد تکرار شده‌اند حذف می‌کند و min_df=2 کلماتی را که کمتر از ۲ بار در کل پیکره متنی تکرار شده‌اند نادیده می‌گیرد. در نهایت، با متد fit_transform() ماتریس ویژگی‌های اسناد محاسبه می‌شود.

۳. **استخراج کلمات کلیدی برتر براساس مجموع امتیازها:** با محاسبه‌ی مجموع امتیازات ماتریس در طول ستون‌ها (tfidf.sum(axis=0)) و مرتب‌سازی نزولی اندیس‌ها با استفاده از np.argsort، ۲۰ کلمه‌ی برتر که بیشترین وزن مجموع را در کل پیکره متنی دارند استخراج می‌شوند.

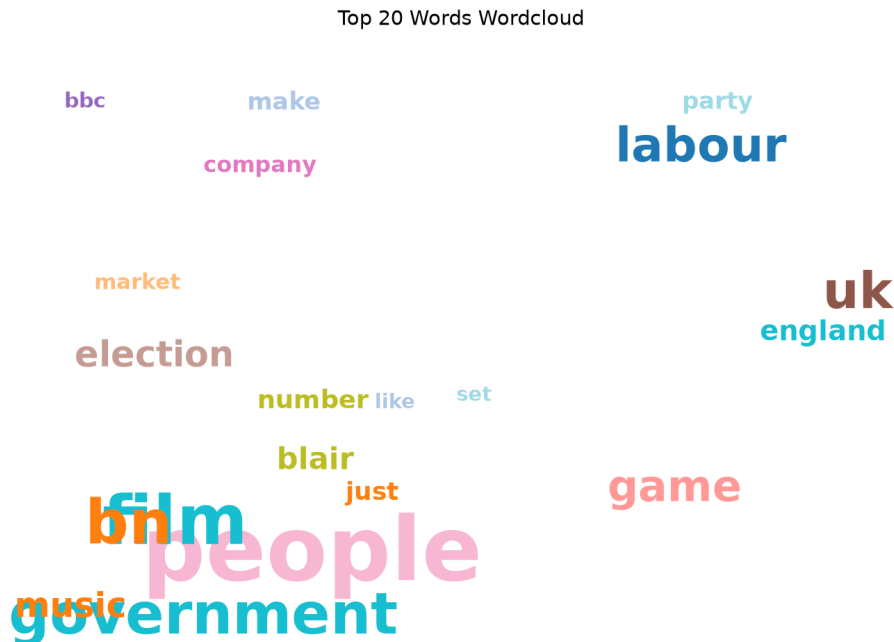
۴. **الگوریتم چیدمان غیرهم‌پوشان (overlaps):** برای جلوگیری از هم‌پوشانی و روی هم افتادن کلمات در تصویر خروجی، تابع کمکی overlaps تعریف شده است. این تابع با تخمین زدن کادر پیرامونی هر کلمه بر اساس طول کلمه و ابعاد فونت، فاصله‌ی موقعیت کلمه‌ی جدید با کلماتی که پیش‌تر جایگذاری شده‌اند را بررسی کرده و در صورت تداخل، موقعیت تصادفی جدیدی را امتحان می‌کند.

۵. شبیه‌سازی ابر کلمات و بهبودهای بصری جدید:

- **مقیاس‌دهی پویا به فونت کلمات:** اندازه‌ی قلم (size) هر کلمه متناسب با فرمول نگاشت خطی بین بازه‌ی ۱۶ الی ۷۱ پیکسل بر اساس امتیاز TF-IDF آن تعیین می‌شود تا کلمات با ارزش‌تر، بزرگ‌تر رندر شوند.
- **رنگ‌آمیزی با پالت متمایز tab20:** رنگ هر کلمه به طور تصادفی از پالت رنگی استاندارد و متمایز tab20 تخصیص می‌یابد تا کنتراست و خوانایی کلمات افزایش یابد.
- **بهبود فونت و چیدمان:** فونت نوشته‌ها به حالت بدون سریف (sans-serif) و ضخیم (bold) تغییر کرده است.

• ذخیره‌سازی پویا: تصویر نهایی با قالب پویای نام فایل خروجی (fname) در مسیر media/ آماده‌ی ذخیره‌سازی است.

۶. تحلیل بصری و بررسی خروجی ابر کلمات جدید: تصویر زیر خروجی نهایی حاصل از اجرای تابع بر روی ۲۰ کلمه‌ی برتر با معیار وزن‌دهی TF-IDF را با پالت رنگی متمایز نشان می‌دهد:



شکل ۴: ابر کلمات اختصاصی رسم‌شده برای ۲۰ کلمه برتر بر اساس امتیاز مجموع TF-IDF

• ابعاد بصری و وزن‌دهی معنایی کلمات: همان‌طور که در شکل ۴ مشخص است، کلماتی نظیر “people” (به رنگ صورتی)، “government” (به رنگ فیروزه‌ای)، “labour” (به رنگ آبی تیره) و “film” با فونت بسیار بزرگ‌تری رندر شده‌اند. این اندازه بزرگ نشان‌دهنده‌ی امتیاز بالا و مجموع وزن TF-IDF بالای آن‌ها در اسناد است. یعنی این واژه‌ها نه تنها تکرار قابل‌قبولی دارند، بلکه به شدت متمایزکننده‌ی موضوعات اصلی (مانند سیاست، سینما و اقتصاد) هستند.

• تمایز موضوعی آشکار: برخلاف نمودارهای فرکانس ساده، در این تصویر کلماتی مانند “election” (انتخابات)، “blair” (بلر - نخست‌وزیر سابق بریتانیا)، “game” (بازی)، “market” (بازار)، “music” (موسیقی) و “company” (شرکت) به وضوح دیده می‌شوند. این توزیع کلمات نشان می‌دهد که پیکره متنی ما به طور مشخص شامل موضوعاتی پیرامون سیاست بریتانیا، اخبار بازار و تجارت، دنیای سرگرمی (موسیقی/بازی/فیلم) و تکنولوژی است.

• تأیید کارکرد فیلترهای بالاگذر و پایین‌گذر: موفقیت الگوریتم در این است که کلمات بسیار عمومی مانند “year”، “new” یا کلمات توقف انگلیسی استاندارد که پیش‌تر در واژه‌نامه‌ی stop_words قرار گرفتند، به کلی فیلتر شده‌اند و در ابر کلمات حضور ندارند. این فرآیند بستری عالی و با کیفیت را برای مرحله‌ی بعدی یعنی ساخت ماتریس برداری اسناد ایجاد می‌کند.

سوال ۱۰ – ساخت ماتریس کیسه کلمات (BoW) با واژگان سفارشی و تقسیم‌بندی داده‌ها
در این مرحله، هدف ساخت ماتریس کیسه کلمات (Bag-of-Words) با استفاده از یک واژگان مشخص واژه‌نامه است که از فایل خارجی words.csv بارگذاری می‌شود. در نهایت داده‌ها به دو بخش مجموعه‌ی کاری (آموزش) و مجموعه‌ی آزمون نگهداشت تقسیم می‌شوند. کد مربوطه به صورت زیر است:

```
1 # =====
2 # Q10 - Bag-of-Words matrix using words.csv vocabulary
3 # =====
4 def q10_bow_matrix(df):
5     words_df = pd.read_csv('words.csv', header=None)
6     vocabulary_list = words_df[0].astype(str).str.lower().tolist()
```

```

7     vectorizer = CountVectorizer(vocabulary=vocabulary_list)
8     bow = vectorizer.fit_transform(df['clean_text'])
9     bow_matrix = bow.toarray()
10
11
12     print(f"\nQ10 | BoW matrix shape: {bow_matrix.shape} (docs x |vocab|={len(
13         vocabulary_list)})")
14
15     return bow_matrix
16
17 def split_2000_225(df, bow_matrix):
18     """Split literally: first 2000 rows = working set, last 225 = held-out test."""
19     df_train = df.iloc[:2000].reset_index(drop=True)
20     df_test = df.iloc[2000:].reset_index(drop=True)
21     bow_train = bow_matrix[:2000]
22     bow_test = bow_matrix[2000:]
23     print("Split | train:", df_train.shape, bow_train.shape,
24         "| test:", df_test.shape, bow_test.shape)
25     print("Split | NOTE: dataset is label-sorted, so the literal last-225 test rows
26         "
27         "are ALL label 4 (Business). See shuffled version in Q20 for a fair "
28         "per-category evaluation.")
29     return df_train, df_test, bow_train, bow_test

```

توضیح عملکرد متدها و تغییرات جدید:

۱. بارگذاری و آماده‌سازی پویا واژگان (Vocabulary):

- `pd.read_csv= ('words.csv', header one)`: واژه‌های مرجع مستقیماً از فایل `words.csv` بارگذاری می‌شوند. از آنجا که این فایل فاقد ردیف هدر (عنوان ستون) است، پارامتر `header=None` تنظیم شده است.
- **یکدست‌سازی واژگان** (`vocabulary_list`): با دستور `astype(str).str.lower().tolist()` تمامی کلمات موجود در ستون اول فایل به نوع رشته تبدیل شده، حروف آن‌ها کوچک می‌شود و در نهایت به یک لیست معمولی پایتون تبدیل می‌گردند. این کار باعث تطابق کامل و بدون خطای واژگان با متون پیش‌پردازش شده می‌شود.

۲. ساخت ماتریس کیسه کلمات (CountVectorizer):

- **محدودسازی ستون‌ها به واژگان سفارشی**: با پاس دادن `vocabulary=vocabulary_list` به کلاس `CountVectorizer`، مدل را موظف می‌کنیم که ماتریس را صرفاً بر اساس کلمات این لیست بسازد. هر کلمه‌ای در اسناد که در این لیست نباشد، نادیده گرفته خواهد شد.
- **تبدیل به آرایه متراکم** (`toarray()`): خروجی متد `fit_transform` یک ماتریس خلوت است که برای انجام راحت‌تر محاسبات جبر خطی بعدی (مانند تجزیه مقادیر منفرد) با متد `toarray()` به یک آرایه دو بعدی استاندارد و متراکم NumPy تبدیل می‌شود. ابعاد این ماتریس برابر با $2225 \times M$ خواهد بود که در آن M تعداد کلمات واژه‌نامه است.

۳. تقسیم‌بندی داده‌ها به مجموعه‌های آموزش و آزمون: تابع `split_2000_225` داده‌های ورودی را به دو بخش افقی تقسیم می‌کند:

- **مجموعه آموزش (train)**: شامل ۲۰۰۰ سطر اول اسناد و بردارهای کیسه کلمات متناظر با آن‌هاست.
- **مجموعه آزمون (test)**: شامل ۲۲۵ سطر پایانی به عنوان داده‌های آزمون مجزا و دست‌نخورده است.
- **نکته مهم توزیع دسته‌ها**: به دلیل مرتب بودن اولیه مجموعه داده بر اساس برچسب‌ها، ۲۲۵ سطر پایانی همگی دارای برچسب شماره ۴ (تجارت) هستند. این تقسیم‌بندی صرفاً یک پیاده‌سازی گام‌به‌گام اولیه است و جابجایی تصادفی داده‌ها (Shuffle) در بخش‌های بعدی برای ارزیابی نهایی به کار گرفته خواهد شد.


```
Q10 | BoW matrix shape: (2225, 53) (docs x |vocab|=53)
```

```
array([[0, 0, 1, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 1, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]], shape=(2225, 53))
```

شکل ۵: نمای کلی از ماتریس فرکانسی کیسه کلمات (BoW) و ابعاد آن در خروجی مفسر

۴. خروجی فرآیند بردارسازی و تحلیل ساختار ماتریس BoW: پس از اجرای تابع، آرایه خروجی و ابعاد به دست آمده به شکل زیر نمایش داده می‌شود:

- **بررسی ابعاد ماتریس** ((shape=(2225, 53)): خروجی نشان می‌دهد که ماتریس حاصل دارای ۲۲۲۵ سطر (تعداد کل اسناد) و ۵۳ ستون است. این ابعاد دقیقاً تایید می‌کند که تعداد واژه‌های استخراج‌شده از فایل words.csv برابر با ۵۳ کلمه کلیدی متمایز بوده است. بنابراین هر سند متنی به یک بردار ویژگی ۵۳ بُعدی تصویر می‌شود.
- **تحلیل ساختار عددی ماتریس خلوت (Sparse Matrix)**: همان‌طور که در آرایه‌ی خروجی شکل ۵ مشهود است، بخش زیادی از درایه‌های ماتریس مقدار صفر (0) دارند. این ویژگی به دلیل ماهیت اسناد متنی است؛ چرا که در هر سند متنی کوتاه، تنها تعداد انگشت‌شماری از واژگان ۵۳ گانه استفاده شده‌اند. حضور مقادیری مانند 1 در درایه‌ها نمایانگر دفعات تکرار آن واژه‌ی خاص از واژه‌نامه در سند متناظر است. این ساختار صفر و یک فرکانسی، پایه‌ی ریاضی مراحل بعدی تحلیل‌های جبر خطی پروژه را شکل می‌دهد.

بخش افراز مجموعه داده (Partitioning Dataset) – تقسیم ساده در مقابل تقسیم تصادفی
در این بخش، هدف تقسیم مجموعه داده به دو بخش آموزش (Train) با ابعاد ۲۰۰۰ نمونه و آزمون (Test) با ابعاد ۲۲۵ نمونه است. برای این کار دو تابع با رویکردهای متفاوت پیاده‌سازی شده است که کد آن‌ها به صورت زیر است:

```
1 # =====
2 # Dataset Partitioning (Simple vs Shuffled Split)
3 # =====
4 def split_2000_225(df, bow_matrix):
5     """Split literally: first 2000 rows = working set, last 225 = held-out test."""
6     df_train = df.iloc[:2000].reset_index(drop=True)
7     df_test = df.iloc[2000:].reset_index(drop=True)
8     bow_train = bow_matrix[:2000]
9     bow_test = bow_matrix[2000:]
10    print("Split | train:", df_train.shape, bow_train.shape,
11          "| test:", df_test.shape, bow_test.shape)
12    return df_train, df_test, bow_train, bow_test
13
14 def split_2000_225_shuffle(df, bow_matrix):
15     df_train, df_test, bow_train, bow_test = train_test_split(
16         df,
17         bow_matrix,
18         train_size=2000,
19         shuffle=True,
20         random_state=42
21     )
22
23     df_train = df_train.reset_index(drop=True)
24     df_test = df_test.reset_index(drop=True)
25
26     print("Split | train:", df_train.shape, bow_train.shape,
27           "| test:", df_test.shape, bow_test.shape)
28     return df_train, df_test, bow_train, bow_test
```

توضیح عملکرد و مقایسه متدها:

۱. تابع تقسیم ترتیبی و ساده (split_2000_225):

- **عملکرد:** این تابع با استفاده از برش زدن ساده ایندکس‌ها (Slicing) با دستور `iloc`، دقیقاً ۲۰۰۰ سطر نخست را به عنوان مجموعه آموزش و ۲۲۵ سطر انتهایی را به عنوان مجموعه آزمون جدا می‌کند.
- **ریست کردن ایندکس:** با متد `reset_index(drop=True)` ایندکس‌های دیتافریم‌های جدید دوباره از صفر مقداردهی می‌شوند تا در تحلیل‌های بعدی خطایی رخ ندهد.
- **اشکال ساختاری (اریب بودن داده‌ها):** از آن‌جا که داده‌های اولیه بر اساس برچسب‌ها مرتب شده‌اند، ۲۲۵ سطر آخر همگی متعلق به کلاس شماره ۴ (Business) خواهند بود. این یعنی مدل هیچ داده‌ی ارزیابی از دسته‌های دیگر (مانند سیاست یا ورزش) در این نوع تقسیم‌بندی نخواهد داشت که منجر به ارزیابی غیرمنصفانه می‌شود.

۲. تابع تقسیم تصادفی مجهز به برهم‌زدن (split_2000_225_shuffle):

- **عملکرد:** این تابع از متد استاندارد `train_test_split` استفاده می‌کند تا فرآیند تقسیم‌بندی را به شکل کاملاً علمی و توزیع‌شده انجام دهد.
- **پارامتر shuffle=True:** قبل از جدا کردن سطرها، موقعیت تمامی نمونه‌ها به صورت تصادفی برهم زده می‌شود. این کار تضمین می‌کند که داده‌های هر دو بخش آموزش و آزمون شامل توزیع یکنواختی از تمامی کلاس‌ها (سیاست، تجارت، ورزش و غیره) باشند.
- **پارامتر random_state=42:** با تعیین هسته تصادفی ثابت (عدد ۴۲)، فرآیند مخلوط‌سازی داده‌ها در هر بار اجرای کد دقیقاً یکسان خواهد بود تا نتایج آزمایش‌ها قابل بازتولید (Reproducible) باشد.

سوال ۱۱ – استانداردسازی داده‌ها و اعمال تجزیه مقادیر منفرد (SVD)

در این بخش، ابتدا داده‌های فرکانسی کیسه کلمات (BoW) مقیاس‌دهی و استانداردسازی می‌شوند تا اثر تفاوت در طول اسناد و فرکانس‌های مطلق واژه‌ها خنثی شود. سپس، با استفاده از الگوریتم تجزیه مقادیر منفرد (SVD)، ماتریس داده‌ها به مؤلفه‌های سازنده‌اش تجزیه می‌گردد. کد این بخش به صورت زیر است:

```
1 # =====
2 # Q11 - Standardization and Singular Value Decomposition (SVD)
3 # =====
4 def q11_standardize_and_svd(bow_matrix):
5     scaler = StandardScaler()
6     bow_scaled = scaler.fit_transform(bow_matrix)
7
8     U, S, Vt = np.linalg.svd(bow_scaled, full_matrices=False)
9
10    print("=== SVD Matrices Dimensions ===")
11    print(f"Original Scaled Matrix (M x N) : {bow_scaled.shape}")
12    print(f"U Matrix (Left Singular)       : {U.shape}")
13    print(f"S Array (Singular Values)        : {S.shape}")
14    print(f"V^T Matrix (Right Singular)      : {Vt.shape}")
15
16    return bow_scaled, U, S, Vt
```

توضیح عملکرد متدها و مفاهیم ریاضی:

۱. **استانداردسازی با StandardScaler:** از آن‌جا که کلمات مختلف فرکانس‌های متفاوتی دارند، واژه‌های بسیار پرتکرار می‌توانند به طور ناعادلانه‌ای روی محاسبات فاصله و ابعاد اثر بگذارند. کلاس `StandardScaler` با کسر میانگین هر ویژگی (ستون) و تقسیم آن بر انحراف معیار، داده‌ها را به گونه‌ای مقیاس‌دهی می‌کند که میانگین آن‌ها برابر با ۰ و واریانس آن‌ها برابر با ۱ شود:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

۲. **تجزیه مقادیر منفرد (SVD) به فرم باریک (full_matrices=False):** رابطه ریاضی حاکم بر این تجزیه به صورت زیر است:

$$A_{M \times N} = U_{M \times K} \Sigma_{K \times K} V_{K \times N}^T$$

که در آن $K = \min(M, N)$ است.

- تنظیم پارامتر `full_matrices=False` فرمت کاهش یافته یا باریک (Reduced/Thin SVD) را پیاده سازی می کند. این کار به جای تولید ماتریس های مربعی بزرگ و غیر ضروری، ابعاد مؤلفه ها را محدود به کوچک ترین بعد ماتریس اصلی (N یا همان ۵۳ بعد واژگان) نگه می دارد تا از هدر رفت حافظه و محاسبات اضافه جلوگیری شود.

۳. تحلیل ابعاد ماتریس های خروجی در ترمینال: با فرض اعمال این تابع روی کل مجموعه داده با ابعاد 2225×53 ، ابعاد ماتریس ها در خروجی به شرح زیر تحلیل می شوند:

- ماتریس استاندارد شده (bow_scaled): ابعاد آن دست نخورده و برابر با (2225, 53) باقی می ماند.
- ماتریس بردارهای منفرد چپ (U): ابعاد آن (2225, 53) خواهد بود که اسناد را در فضای پنهان (Latent Space) توصیف می کند.
- آرایه مقادیر منفرد (S): این آرایه تک بعدی شامل ۵۳ مقدار منفرد مرتب شده به صورت نزولی است که نشان دهنده میزان پراکندگی یا اهمیت هر یک از محورهای مختصات جدید است.
- ماتریس بردارهای منفرد راست (Vt): ابعاد آن (53, 53) خواهد بود که کلمات را در فضای پنهان توصیف کرده و ارتباط واژگان با ویژگی های پنهان را مشخص می کند.

۴. خروجی فرآیند تجزیه SVD و تحلیل ابعاد ماتریس ها: پس از اجرای تابع بر روی داده های استاندارد شده مجموعه آموزش (که شامل ۲۰۰۰ سند است)، ابعاد واقعی ماتریس های حاصل به شکل زیر در خروجی ظاهر می شود:

```
=== SVD Matrices Dimensions ===
Original Scaled Matrix (M x N) : (2000, 53)
U Matrix (Left Singular)       : (2000, 53)
S Array (Singular Values)      : (53,)
V^T Matrix (Right Singular)    : (53, 53)
```

شکل ۶: ابعاد چاپ شده برای ماتریس های تجزیه SVD در خروجی محیط پایتون

- ماتریس مقیاس شده اصلی (Original Scaled Matrix): ابعاد این ماتریس برابر با (2000, 53) است. این نشان می دهد که تجزیه بر روی بخش آموزش داده ها (با ۲۰۰۰ سند و ۵۳ ویژگی کلیدی) اعمال شده است.
- ماتریس U (بردارهای منفرد چپ): ابعاد این ماتریس برابر با (2000, 53) است. به دلیل تنظیم حالت باریک (`full_matrices=False`)، تعداد ستون های این ماتریس به جای ۲۰۰۰ (در حالت SVD کامل)، برابر با تعداد کلمات واژه نامه (یعنی ۵۳) محدود شده است که هر سند را در فضای ویژگی های پنهان جدید نمایش می دهد.
- آرایه S (مقادیر منفرد): ابعاد این آرایه به صورت تک بعدی با طول ۵۳ رندر شده است ((53,)). این ۵۳ مقدار مثبت و نزولی، میزان واریانس و پراکندگی داده ها را روی تک تک مؤلفه های جدید نشان می دهند.
- ماتریس V^T (بردارهای منفرد راست): ابعاد آن (53, 53) است که ارتباط مستقیم بین ۵۳ کلمه ی واژه نامه اصلی و ۵۳ بعد پنهان جدید کشف شده توسط الگوریتم را فرموله سازی می کند.

سوال ۱۲ – تعیین پویای آستانه حفظ اطلاعات، ترسیم نمودار و اعمال Truncated SVD
در این گام، هدف تعیین ساختاریافته و خودکار بعد بهینه (k) برای کاهش بعد داده‌ها است. به جای حدس شهودی، بعد بهینه دقیقاً بر اساس آستانه‌ی ریاضی حفظ ۹۰ درصد واریانس کل داده‌ها محاسبه می‌شود. کد بهینه‌شده به صورت زیر است:

```

1 # =====
2 # Q12 - Scree plot + threshold selection + Truncated SVD + error
3 # =====
4 def q12_truncated_svd(bow_scaled, U, S, Vt, fname, fname2, variance_threshold=0.90)
5 :
6     total_variance = np.sum(S ** 2)
7     cumulative_variance = np.cumsum(S ** 2) / total_variance
8
9     k_optimal = np.argmax(cumulative_variance >= variance_threshold) + 1
10
11     fig, ax = plt.subplots(figsize=(10, 6))
12
13     ax.plot(cumulative_variance, color='steelblue', linewidth=2)
14     ax.axhline(y=variance_threshold, color='red', linestyle='--', label=f'Threshold
15         ({variance_threshold*100}%)')
16     ax.vline(x=k_optimal, color='green', linestyle='--', label=f'Optimal k = {
17         k_optimal}')
18
19     ax.set_title('Cumulative Explained Variance by Singular Values', fontsize=14)
20     ax.set_xlabel('Number of Singular Values (k)', fontsize=12)
21     ax.set_ylabel('Cumulative Variance', fontsize=12)
22     ax.legend(loc='lower right')
23     plt.grid(True, alpha=0.3)
24     plt.tight_layout()
25     plt.savefig(f'media/{fname}', dpi=130)
26     plt.close(fig)
27
28     print(f"Suggested k to retain {variance_threshold*100}% variance: {k_optimal}")
29
30     U_truncated = U[:, :k_optimal]
31     S_truncated = np.diag(S[:k_optimal])
32     Vt_truncated = Vt[:k_optimal, :]
33
34     print("\n=== Truncated Matrices Dimensions ===")
35     print(f"U_k shape : {U_truncated.shape}")
36     print(f"S_k shape : {S_truncated.shape}")
37     print(f"Vt_k shape : {Vt_truncated.shape}")
38
39     bow_reconstructed = np.dot(U_truncated, np.dot(S_truncated, Vt_truncated))
40
41     reconstruction_error = np.linalg.norm(bow_scaled - bow_reconstructed, 'fro')
42     print(f"Reconstruction Error : {reconstruction_error:.4f}")
43
44     return U_truncated, S_truncated, Vt_truncated, bow_reconstructed

```

توضیح عملکرد متدها و منطق ریاضی:

- محاسبه‌ی واریانس تجمعی توجیه‌شده (cumulative_variance):** در تجزیه مقادیر منفرد، مجذور هر مقدار منفرد (s_i^2) نشان‌دهنده‌ی میزان واریانس (اطلاعات) حفظ‌شده روی آن بعد پنهان است.
 - ابتدا کل واریانس با جمع مجذور تمامی مقادیر منفرد محاسبه می‌شود: $\sum s_i^2$.
 - با تقسیم مجموع تجمعی مجذورات بر واریانس کل، نسبت واریانس تجمعی توجیه‌شده به دست می‌آید که مقداری در بازه‌ی $[0, 1]$ است.

- یافتن خودکار بعد بهینه (k):** با دستور `np.argmax(cumulative_variance >= variance_threshold)` نخستین اندیسی که در آن نسبت واریانس تجمعی از آستانه تعریف شده (مثلاً ۹۰٪ یا همان ۰.۹۰) عبور می‌کند به طور

خودکار پیدا می‌شود. با اضافه کردن عدد ۱ به این اندیس، مقدار بهینه‌ی k به دست می‌آید.

۳. رسم تخصصی نمودار تجمعی و خطوط راهنما:

- خط افقی نقطه‌چین قرمز رنگ (red dashed line) آستانه مورد نیاز (مثلاً ۹۰٪) را روی محور عمودی تصویر می‌کند.
- خط عمودی نقطه‌چین سبز رنگ (green dashed line) مقدار دقیق k بهینه محاسباتی را روی محور افقی نشان می‌دهد تا نقطه تلاقی آن‌ها کاملاً ملموس باشد.
- تصویر نهایی با نام پویا از طریق متغیر fname در مسیر مشخص‌شده ذخیره می‌شود.

۴. کوتاه‌سازی ماتریس‌ها (SVD Truncated): پس از تعیین مقدار پویای k ، ماتریس‌ها بر اساس این بعد جدید برش داده می‌شوند:

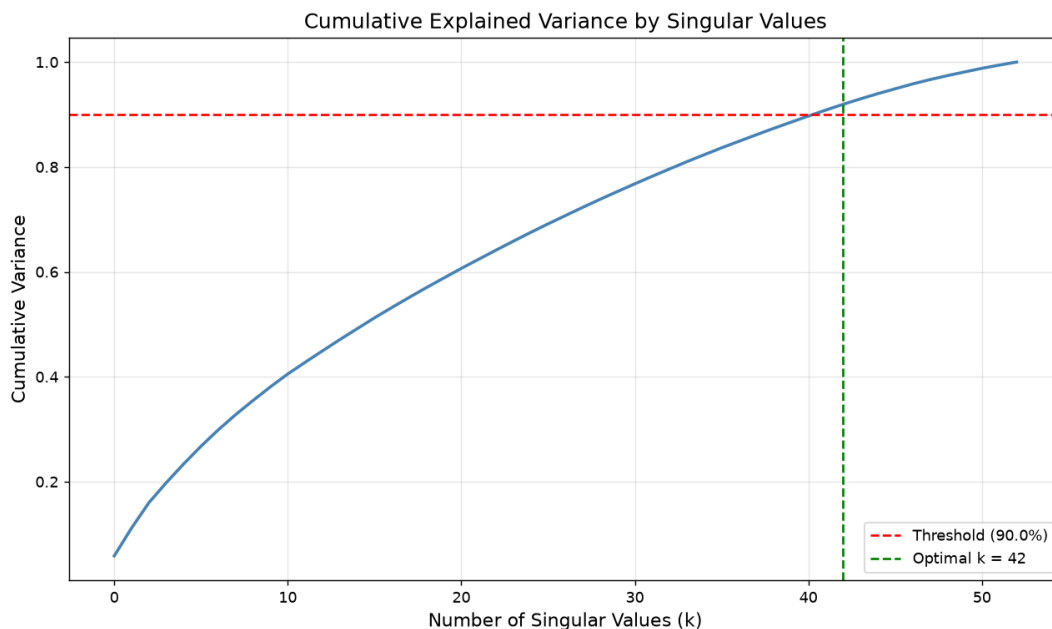
- $U_truncated$: صرفاً k بعد پنهان برتر برای اسناد نگه داشته می‌شود.
- $S_truncated$: آرایه مقادیر منفرد برتر به یک ماتریس قطری با ابعاد $k \times k$ تبدیل می‌شود.
- $Vt_truncated$: صرفاً فرکانس و وزن کلمات روی k بعد پنهان اول حفظ می‌شود.

۵. بازسازی ماتریس و خطای فروبنیوس: با ضرب متوالی ماتریس‌های کوتاه‌سازی‌شده (np.dot)، ماتریس تقریب‌زده شده با رتبه‌ی پایین ($bow_reconstructed$) تشکیل می‌شود:

$$A_k = U_k \cdot \Sigma_k \cdot V_k^T$$

سپس تفاوت میان ماتریس استاندارد شده‌ی اولیه و ماتریس بازسازی‌شده با استفاده از نرم فروبنیوس (Frobenius Norm) به عنوان معیار خطای بازسازی محاسبه می‌گردد. هر چه مقدار k بزرگ‌تر باشد، این خطا به صفر نزدیک‌تر می‌شود، اما بعد فضای ویژگی نیز بزرگ‌تر خواهد شد.

۶. تحلیل تجربی نمودار واریانس تجمعی و تعیین بعد بهینه: پس از اجرای الگوریتم بر روی داده‌های استاندارد شده‌ی مجموعه آموزش، نمودار تعیین آستانه و فرآیند کاهش بعد به شکل زیر ترسیم و ذخیره می‌شود:



شکل ۷: نمودار واریانس تجمعی توجیه‌شده به همراه خطوط راهنما برای تعیین بعد بهینه k

- **تحلیل منحنی واریانس تجمعی (Cumulative Variance):** منحنی آبی‌رنگ نشان می‌دهد که با اضافه شدن هر یک از مقادیر منفرد (k)، چه میزان از اطلاعات (واریانس) کل داده‌ها پوشش داده می‌شود. شیب صعودی منحنی در ابتدا بسیار تیز است که نشان‌دهنده تمرکز شدید اطلاعات در مؤلفه‌های پنهان اولیه است. با حرکت به سمت راست، شیب منحنی به تدریج ملایم‌تر می‌شود که رفتاری کاملاً طبیعی در تجزیه مقادیر منفرد است.

• **تعیین بعد بهینه ($k = 42$):** مبنای تصمیم‌گیری برای کاهش بعد، حفظ حداقل ۹۰ درصد از واریانس کل داده‌ها تعریف شده است:

– خط افقی نقطه‌چین قرمز رنگ (red dashed line) مرز بحرانی ۹۰ درصد (0.90) را روی محور عمودی مشخص می‌کند.

– خط عمودی نقطه‌چین سبز رنگ (green dashed line) دقیقاً در نقطه‌ای رسم شده است که منحنی آبی‌رنگ از این مرز عبور می‌کند.

– بر اساس محاسبات دقیق‌تر، در نقطه $k = 42$ میزان واریانس تجمعی توجیه‌شده از ۹۰ درصد فراتر می‌رود. بنابراین، سیستم به طور خودکار بعد فضا را از ۵۳ ویژگی اولیه به ۴۲ مؤلفه پنهان کاهش می‌دهد.

• **نتیجه‌گیری عملی از کاهش بعد:** انتخاب $k = 42$ نشان می‌دهد که با حذف ۱۱ بعد کم‌اهمیت‌تر (که حدود ۲۰٪ از ابعاد کل واژه‌نامه است)، همچنان بیش از ۹۰٪ از واریانس و اطلاعات ساختاری داده‌ها حفظ می‌شود. این فشرده‌سازی هوشمندانه علاوه بر کاهش حجم محاسبات، با حذف نویزهای موجود در فرکانس‌های ضعیف کلمات، دقت مدل‌های دسته‌بندی بعدی را بهبود می‌بخشد.